Desafio Técnico: Projeto Pokédex

Refatoração de Interface e Arquitetura MVC com JSON Dinâmico e Migração de Páginas Estáticas para Template Unificado Roteado em PHP.

Objetivo do Desafio

Você recebeu uma estrutura de páginas responsivas estáticas focadas em criaturas do jogo Pokémon Unite (como `2.azumarill.html` e `18.charizard.html`), além de uma base de dados centralizada no arquivo físico `pokemon.json`.

O seu desafio como desenvolvedor Back-End é **eliminar completamente a redundância de arquivos HTML estáticos individuais** e unificar a renderização da aplicação em um **único arquivo de template dinâmico (**`pokemon.php`**)**. As requisições de consulta devem bater obrigatoriamente no Front Controller (`index.php`), mapeando o arquivo de dados bruto (`pokemon.json`) de forma transparente e segura através das funções globais e do motor contidos no arquivo `config.php`.

1. Mapeamento do Ecossistema Arquitetural

Toda a infraestrutura original de páginas soltas e folhas de estilo parciais deve ser migrada e organizada conforme a divisão de responsabilidades da arquitetura MVC:

```
/ projpokedex
├── /controllers               <-- Camada de Controle
│   ├── HomeController.php     <-- Instancia e renderiza a lista da vitrine
│   ├── PaginaController.php   <-- Controlador Coringa (Fallback)
│   └── PokemonController.php  <-- Captura slugs/IDs e a exibição do card
├── /log                       <-- Destino de logs gerados pelo sistema
├── /models                    <-- Camada de Dados (Models / Conexão)
│   └── PokemonRepository.php  <-- lê o pokemon.json e filtrar por ID ou Slug
├── /views                     <-- Camada Visual Fragmentada em PHP
│   ├── /assets                <-- Recursos Estáticos
│   │   ├── /css
│   │   │   ├── global.css     <-- Regras de estilo e fontes tipográficas
│   │   │   ├── index.css      <-- Regras específicas de layout da Home Page
│   │   │   └── pokemon.css    <-- Folha estrutural p/ injeção dinâmica de temas
│   │   ├── /img          <-- Banco de evoluções, ícones de estatísticas e logos
│   │   └── /js
│   │       ├── index.js  <-- Scripts de comportamento estático da interface
│   │       └── pokemon.json   <-- Arquivo com as estatísticas e metadados
│   ├── /includes
│   │   ├── footer.php    <-- Rodapé unificado de encerramento do contêiner
│   │   └── header.php    <-- Cabeçalho Global dinâmico contendo a Navbar
│   ├── 404.php           <-- View Dedicada para Rotas Inválidas
│   ├── home.php          <-- Vitrine inicial com os cards de linkagem rápida
│   └── pokemon.php       <-- TEMPLATE ÚNICO p/ renderização dinâmica de dados
├── config.php            <-- Helpers, Template Engine e Autoload de Classes
└── index.php             <-- Front Controller (roteamento)
```

2. Infraestrutura Fornecida e Roteamento Parametrizado

A aplicação deverá se apoiar estritamente na mecânica de Front Controller unificado e no Template Engine fornecido pelos arquivos centrais. O formato esperado para as URLs amigáveis de navegação obedecerá às seguintes regras lógicas:

1. **Acesso à Página Inicial (Vitrine):** `index.php?route=home` ou puramente `index.php`. O Front Controller invocará automaticamente o método padrão do `HomeController`.
2. **Acesso Dinâmico ao Card de Atributos (index.php?route=pokemon_detalhes&id=18)**: O Front Controller instanciará a classe `PokemonController` e despachará o fluxo para o método `detalhes()`, capturando o parâmetro numérico enviado via URL.

3. Regras de Implementação Obrigatórias

Para obter aprovação na entrega do projeto, devem ser cumpridas as seguintes diretivas técnicas de codificação:

- **Modularização dos Fragmentos Globais (header.php / footer.php)**: A barra de navegação principal e as tags estruturais abertas devem ser extraídas. Os links dos menus devem chamar os endpoints dinâmicos (exemplo: `<a href="<?= BASE_URL ?>/index.php?route=pokemon_detalhes&id=2">Azumarill</a>`). A inclusão das folhas de estilo deve invocar as funções auxiliares `css('global')` e `css('pokemon')` configuradas nas constantes.
- **Injeção de Classes de Estilo Dinâmicas no Escopo:** A folha de estilo `pokemon.css` utiliza regras específicas associadas a IDs ou classes do elemento raiz para alternar o esquema de cores e planos de fundo baseados no tipo do pokémon (ex: fogo, água). O grupo deve injetar dinamicamente a propriedade identificadora diretamente no escopo útil da tag body da View (ex.: `<body class="pokemon-type-<?= e($pokemon['tipo']) ?>">`) para que a interface reflita o tema correto sem duplicação de CSS.
- **Construção da Camada Model via JSON Parser:** A classe `PokemonRepository.php` deve simular a persistência de um banco de dados relacional. Ela deve abrir o arquivo bruto `pokemon.json`, decodificar seus dados nativamente através da instrução `json_decode(file_get_contents(...), true)` e implementar métodos de filtragem lógica (como `getPokemonById($id)` ou `getAll()`).
- **Unificação Absoluta do Template (views/pokemon.php):** Fica expressamente proibido manter os arquivos estáticos `2.azumarill.html` e `18.charizard.html` no servidor. Toda a estrutura visual (seções de nível de poder, barra de estatísticas de ataque/defesa, linha evolutiva e imagens) deve consumir variáveis dinâmicas PHP desempacotadas pela função `render_view()` através do vetor `$data`.

4. Critérios de Avaliação e Segurança Ativa

- **Proteção Cross-Site Scripting (XSS) na Camada View**: Quaisquer saídas textuais ou strings alfanuméricas recuperadas do arquivo JSON que forem ecoadas nas interfaces HTML devem obrigatoriamente passar pela função helper de higienização `e()`, impedindo a injeção e execução de scripts maliciosos.

- **Tratamento de Exceções Baseado em Fallback de Erro:**
    - Se a URL tentar acessar uma rota maliciosa ou modificadores inválidos na query string, o Front Controller disparará o fallback para o `PaginaController`, que renderizará a tela de erro `404.php`.
    - Se a rota do controlador for válida (`pokemon_detalhes`), mas o parâmetro identificador (`&id=`) enviado via URL não for localizado dentro das chaves de dados do arquivo `pokemon.json`, o método do controlador deve interromper a execução e acionar explicitamente a renderização da view `render_view('404')`.
- **Fidelidade Arquitetural e Preservação de Interface**: A separação de responsabilidades nas pastas deve ser mantida de forma estrita. A responsividade e o comportamento reativo móvel (Media Queries) originais das folhas de estilo fornecidas não podem sofrer quebras estruturais ou deformações visuais após a conversão dos arquivos dinâmicos em componentes PHP.